

DAY 2 / 4

Day 2

주제 · **컨텍스트**

- ① 의도 전달
- ② 의도 추출
- ③ 컨텍스트 최적화

DAY 2

컨텍스트 엔지니어링

AI가 원하는 결과를 내지 못하는 이유

프롬프트 엔지니어링



컨텍스트 엔지니어링

"컨텍스트 엔지니어링은 컨텍스트 윈도우를 **딱 맞는 정보로 채우는 섬세한 기술이자 과학"**

— Andrej Karpathy, 2025

2025년 Karpathy · Tobi Lütke 등이 용어를 대중화 → **업계 표준 관점**으로 자리잡음

**왜 AI는 내가 원하는 결과를
내지 못하는가?**

컨텍스트 엔지니어링 · 결과를 못 내는 이유 2가지



01



의도·정보 전달 미흡

AI

완료 기준 등이 명확히 정의되지 않아 스스로 판단함

02



AI 역량 부족

AI

AI 역량의 한계

👉 경험상 문제의 대부분은 의도·정보 전달 미흡 등으로 발생

컨텍스트 엔지니어링 · 오늘 다룰 주제



01



의도 전달

다양한 방법으로 AI에게 의도·정보 전달하기

02



의도 추출

AI 도움으로 내 의도 더 구체적으로 정리하기

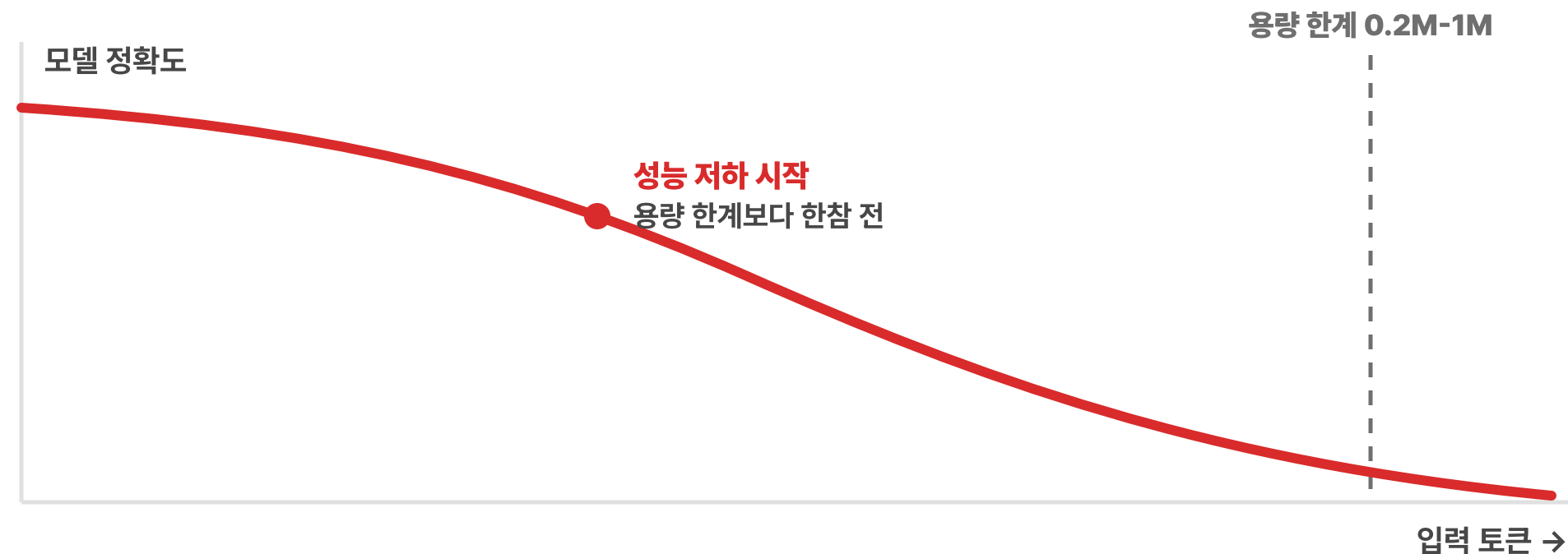
03



컨텍스트 최적화

제한된 자원인 컨텍스트 최적화하기

컨텍스트 엔지니어링 · 리마인드: Context Rot 문제



🔄 세션은 매 턴 누적 · 직전 모든 메시지를 다시 포함

🎯 긴 컨텍스트는 중간 정보를 놓침 (Lost in the Middle)

컨텍스트는 무한 저장소가 아니라 **쓸수록 썩는 유한 자원** · 다 넣지 말고 의도를 골라 전달

DAY 2

의도 전달

왜 · 무엇 · 어떻게 · 확인

의도 전달 · 의도 전달 개요



01



전달할 내용 준비

02



최적의 전달 방법 결정

03



전달한 내용을
AI가 이해했는지 확인

의도 전달 · 컨텍스트



1회성

1회성 컨텍스트

매번 변함

"버그를 고친 후, E2E 테스트를 하고, 통과하면 커밋해줘"

영구 후보

"**버그를 고치기 전에 Red 테스트를 먼저 만들어줘,** 버그를 고친 후..."

지속

잘 변하지 않는 컨텍스트

세션과 무관하게 유지

정보	
디렉토리 구조	추론 가능
최신 데이터	추론 불가능

지침

약어 등 이해하기 어려운 단어 쓰지 않기

코드 작성 시 테스트 코드 반드시 함께

💡 진한 부분은 **영구 컨텍스트의 후보** - 이런 것을 자동으로 승격시켜준다면 좋을 것

의도 전달 · 잘 변하지 않는 컨텍스트 예시



잘 변하지 않는 컨텍스트는 보통 **md** 파일로 관리 - 종류별 대표 예시

01



기술 지침

`architecture.md · code-style.md · testing.md`

02



도메인 지식

`builder.md · payment.md`

03



비즈니스 방향

`vision.md · business.md`

전달·최적화의 4분류 · Write · Select · Compress · Isolate

전달법·최적화 전략을 묶는 4가지 행위 - 업계가 널리 채택한 실무 분류 LangChain · Context Engineering for Agents · 2025

01



Write · 쓰기

컨텍스트 윈도우 밖에 적어두고 필요할 때 가져옴

CLAUDE.md · 스킬·에이전트 md

02



Select · 고르기

필요한 것만 골라 컨텍스트 윈도우 안으로

lazy 로딩 · rules 매칭 · RAG

03



Compress · 압축

들어온 내용을 작게 만들어 자리 절약
출력 토큰 줄이기 · 요약 · /clear

04



Isolate · 격리

컨텍스트를 나눠 서로 안 섞이게
서브에이전트 위임 · phase 분해

👉 뒤에 나올 7가지 전달법과 7가지 최적화 전략은 모두 이 4개의 인스턴스

의도 전달 · 다양한 전달 방법 · 개요



컨텍스트를 AI에 전달하는 방법 7가지 소개 - 간단한 방식부터 복잡한 방식까지 스펙트럼을 이룸.

간단한 방식

복잡한 방식





01
매번 프롬프트



02
CLAUDE.md 전역



03
md에 포함



04
Lazy 로딩



05
rules 매칭



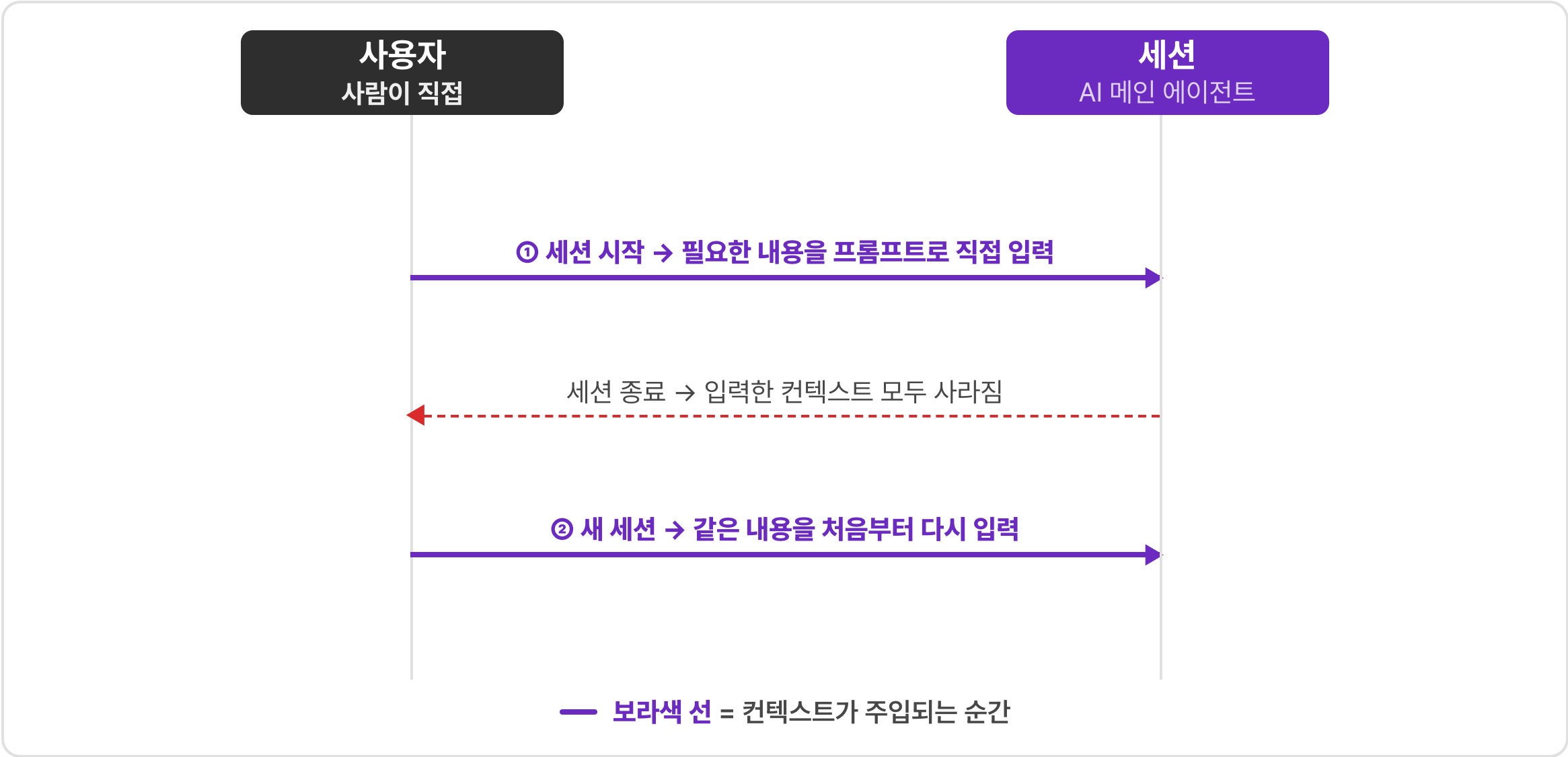
06
pre-hook



07
RAG

👉 이런 방법이 있다는 것만 알아두고, 실제 활용 시 AI와 논의해 결정하면 됨

의도 전달 · 매번 프롬프트로 설명



“ 필요할 때마다 대화창에 직접 설명을 적어 전달하는, 가장 원초적인 방식.

장점

가장 쉬움

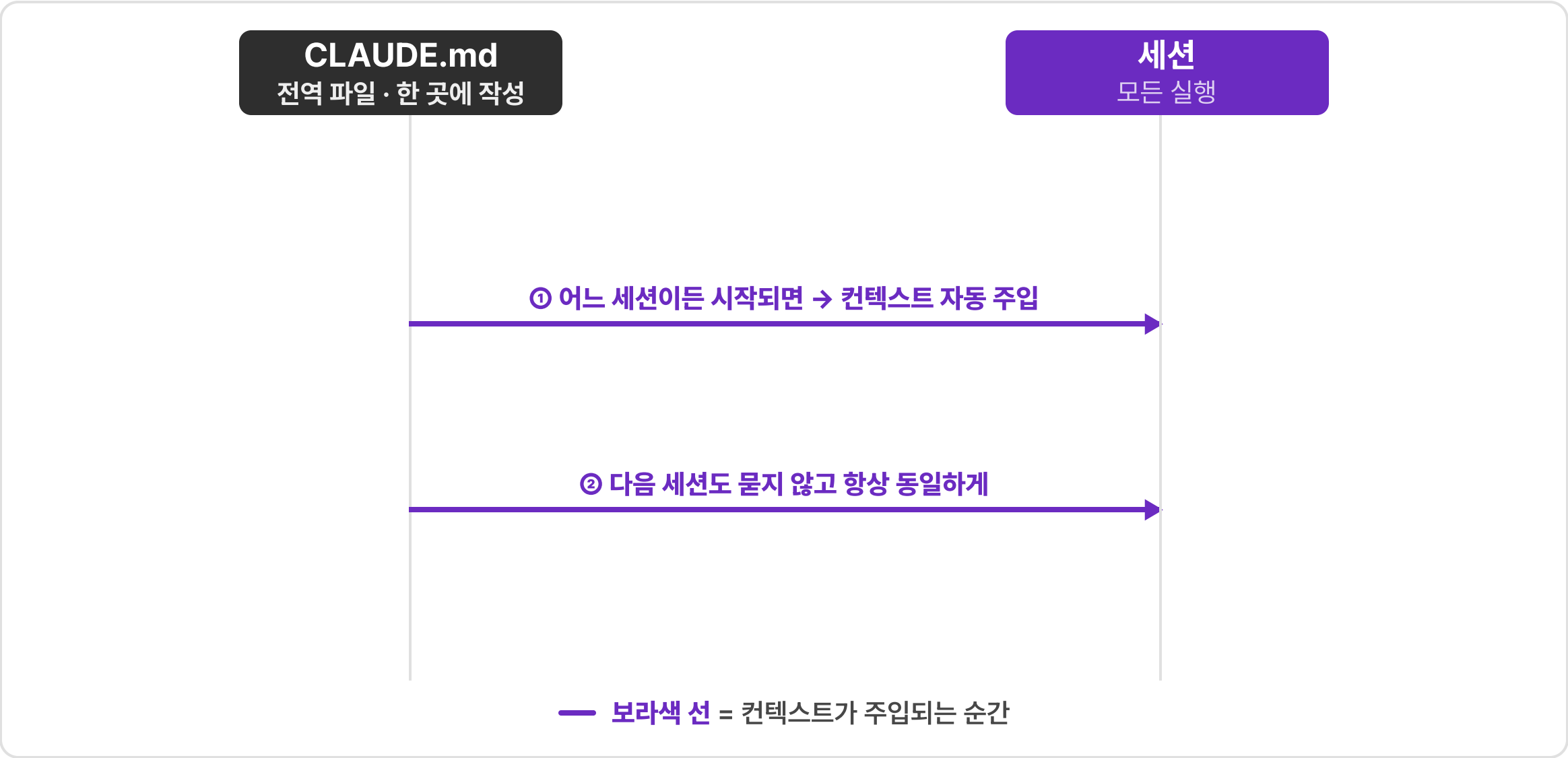
아무 준비 없이 곧바로 시작할 수 있음

단점

매번 새로 설명해야 함

세션이 바뀌면 사라져 반복 노동이 됨

의도 전달 · CLAUDE.md 전역 주입

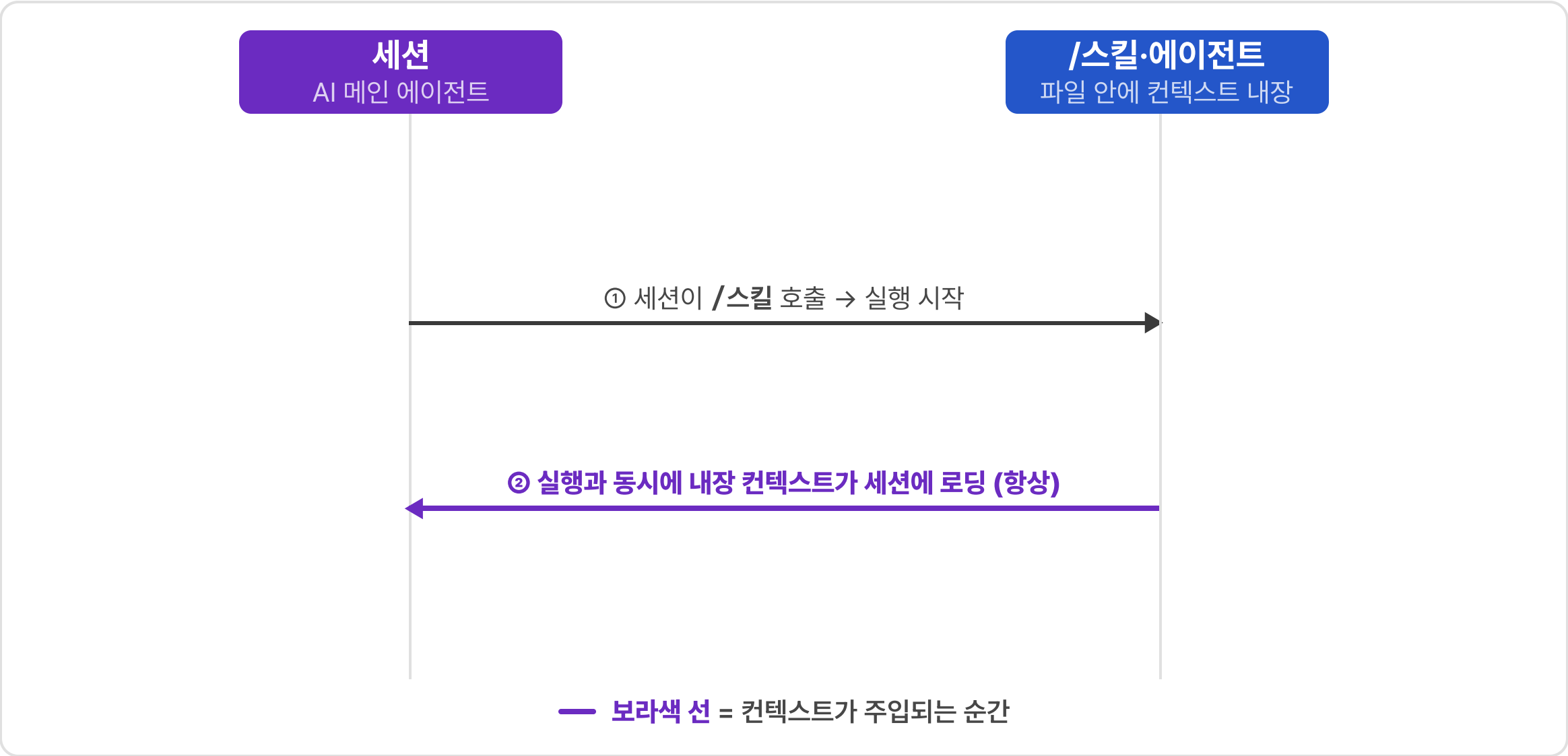


“ 프로젝트 루트의 **CLAUDE.md**에 적어두면 모든 세션에 자동 포함 - **전역 변수**처럼 동작함.

장점
주입이 쉬움
한 번 적어두면 전역 변수처럼 어디서나 적용

단점
모든 세션에 기본 포함
상황별 구분 없이 전역 지침만 활용 가능

의도 전달 · 스킬·에이전트 md에 포함



“ 특정 스킬·에이전트 파일 안에 컨텍스트를 박아두어, 그 스킬이 실행될 때 함께 로딩되게 함.

장점

결정적으로 로딩됨

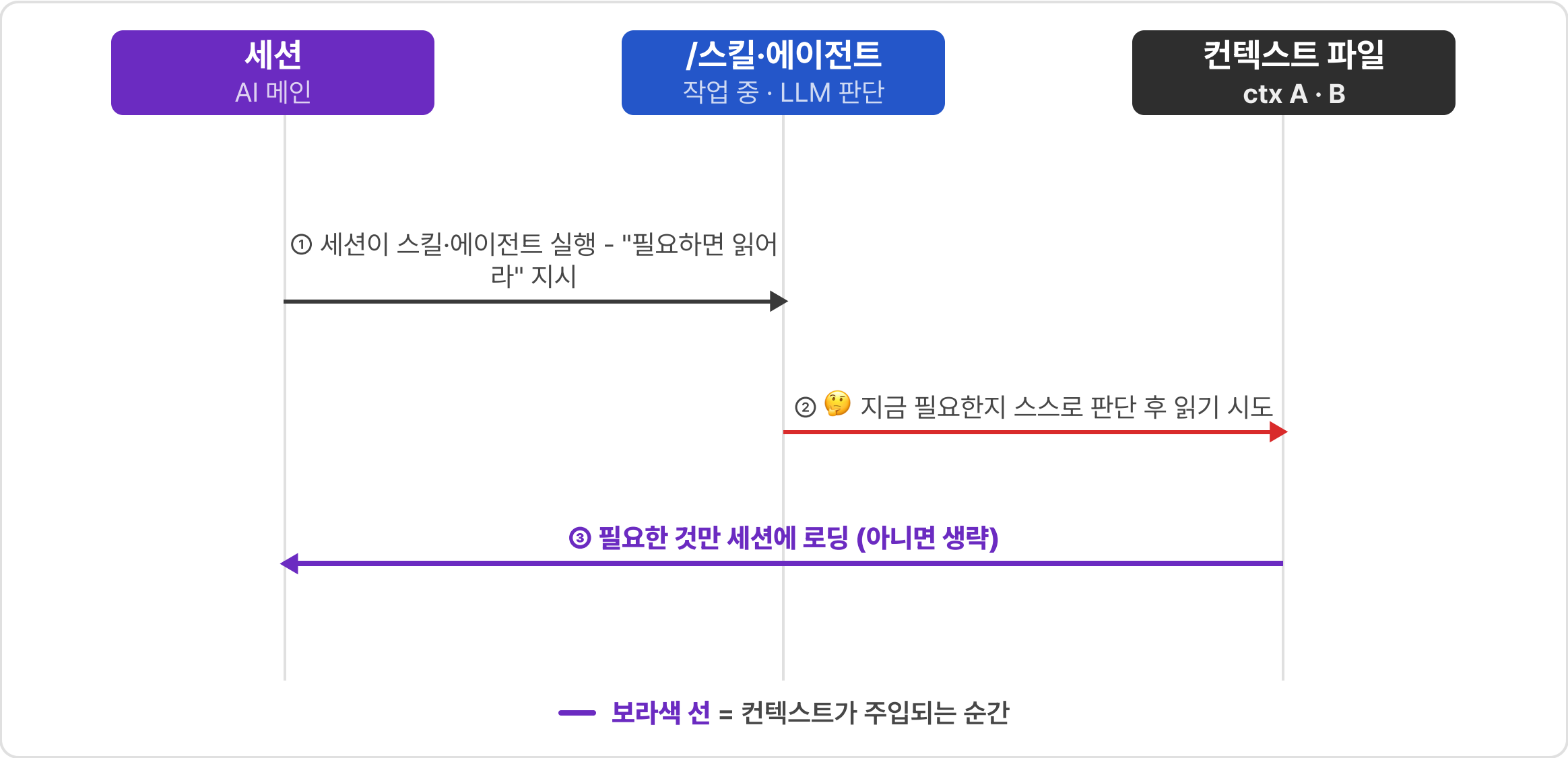
스킬을 부르면 항상 함께 - 동작이 블랙박스로 보장

단점

가시성 낮음 · 재활용 어려움

파일에 묻혀 사람이 보기 어렵고 다른 곳에 쓰기 힘들

의도 전달 · 스킬·에이전트 지시로 Lazy 로딩



“ 스킬 안에서 “필요하면 X를 읽어라”처럼 지시해, LLM이 필요한 순간 판단 해 불러오게 함.

장점

필요한 것만 지능적 로딩

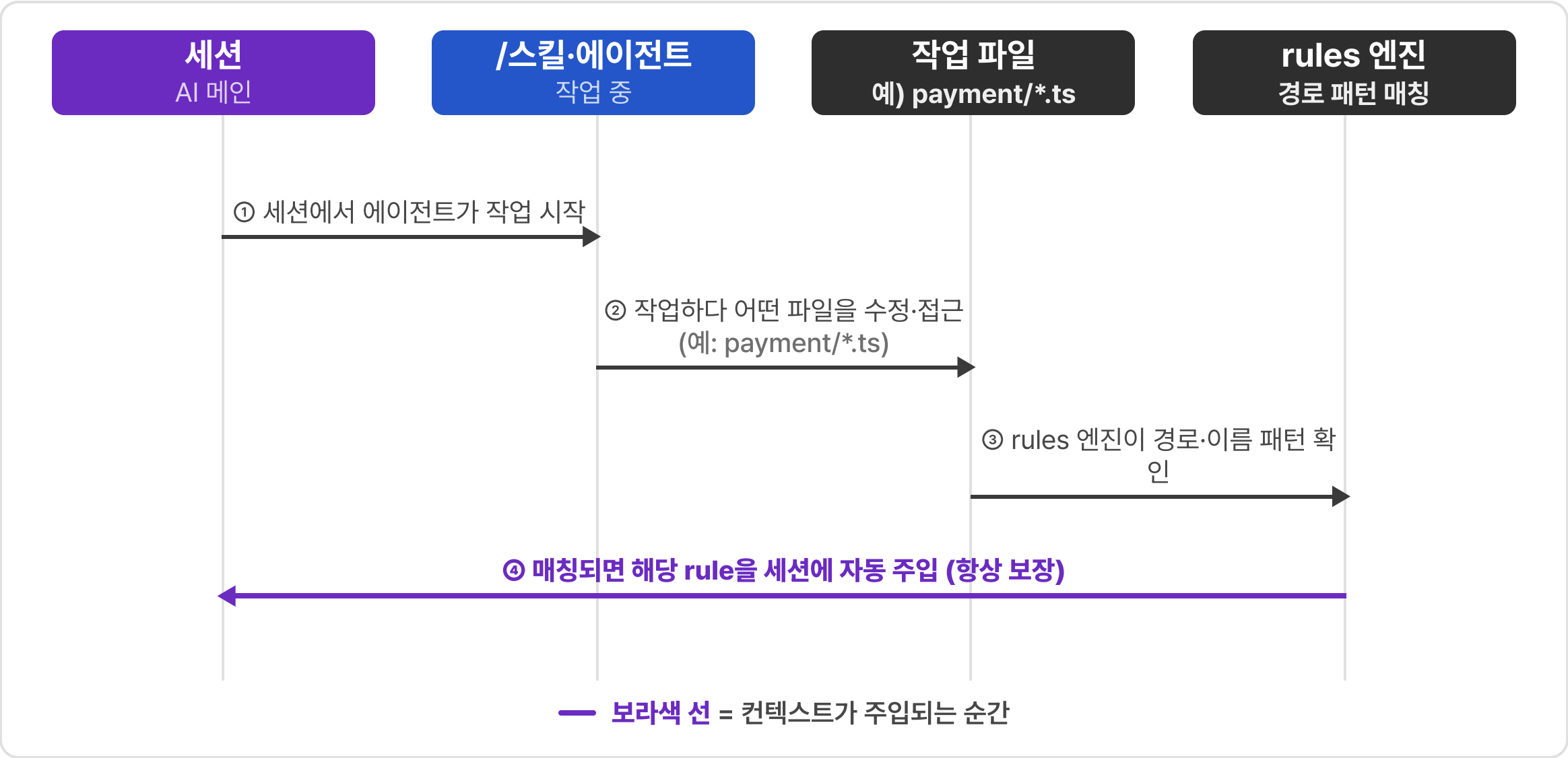
LLM이 그때그때 판단해 꼭 필요한 컨텍스트만 가져옴

단점

느림 · 로딩 보장 안 됨

판단에 시간이 걸리고, 비결정성으로 빠뜨릴 수 있음

의도 전달 · .claude/rules 파일명 매칭



“ 파일 경로·이름 **패턴에 매칭** 되면 해당 규칙을 자동 주입하는, 규칙 기반 정적 주입 방식.

장점

빠름 · 로딩 보장됨

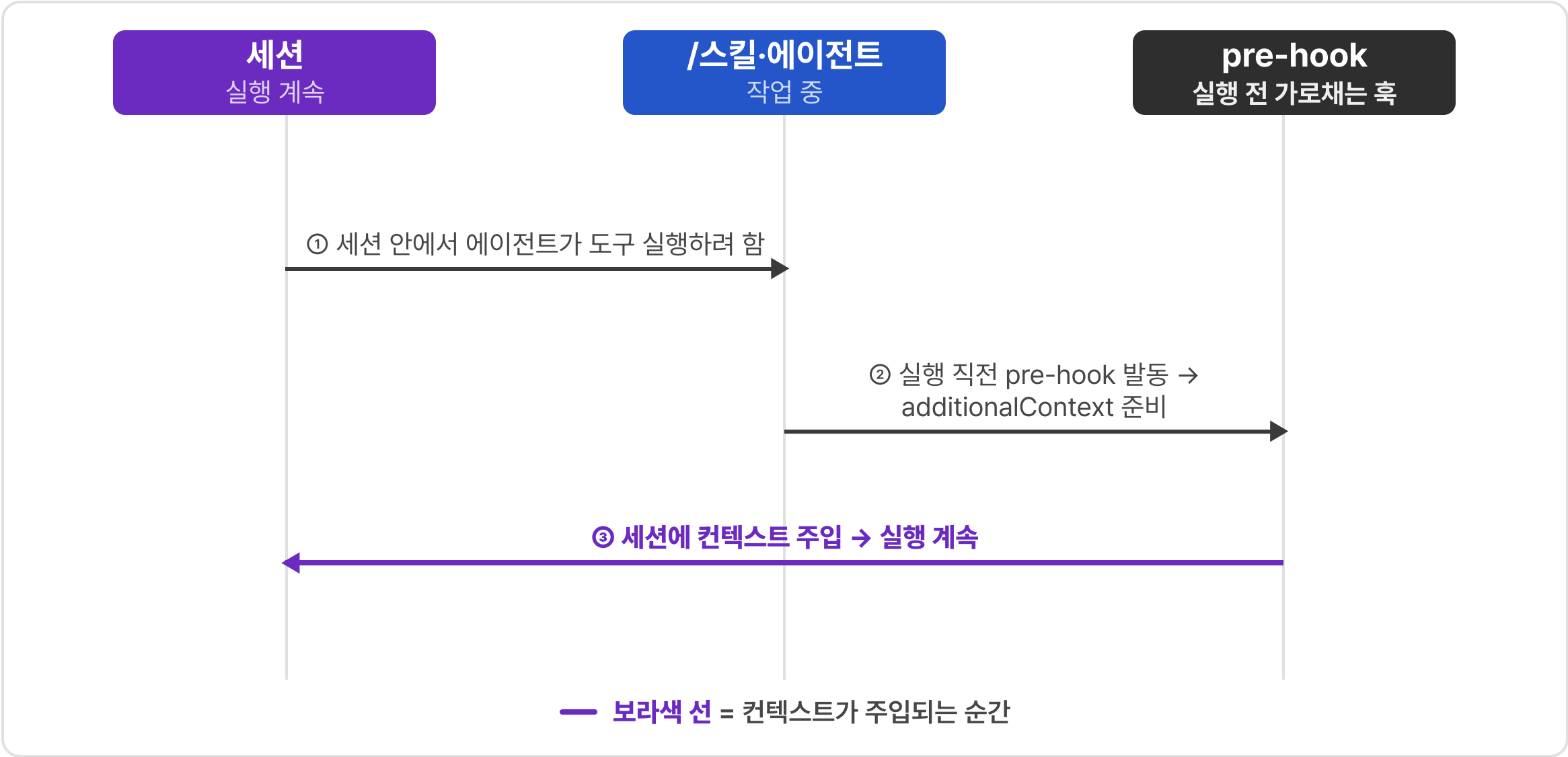
패턴만 맞으면 즉시·확실하게 주입됨

단점

정교한 구분 어려움

정적 규칙이라 상황에 따른 세밀한 구분이 힘들

의도 전달 · pre-hook additionalContext



“ 도구 실행 직전 훅에서 additionalContext로 컨텍스트를 끼워 넣는 방식 - **특성은 rules와 동일** .

장점

빠름 · 로딩 보장됨

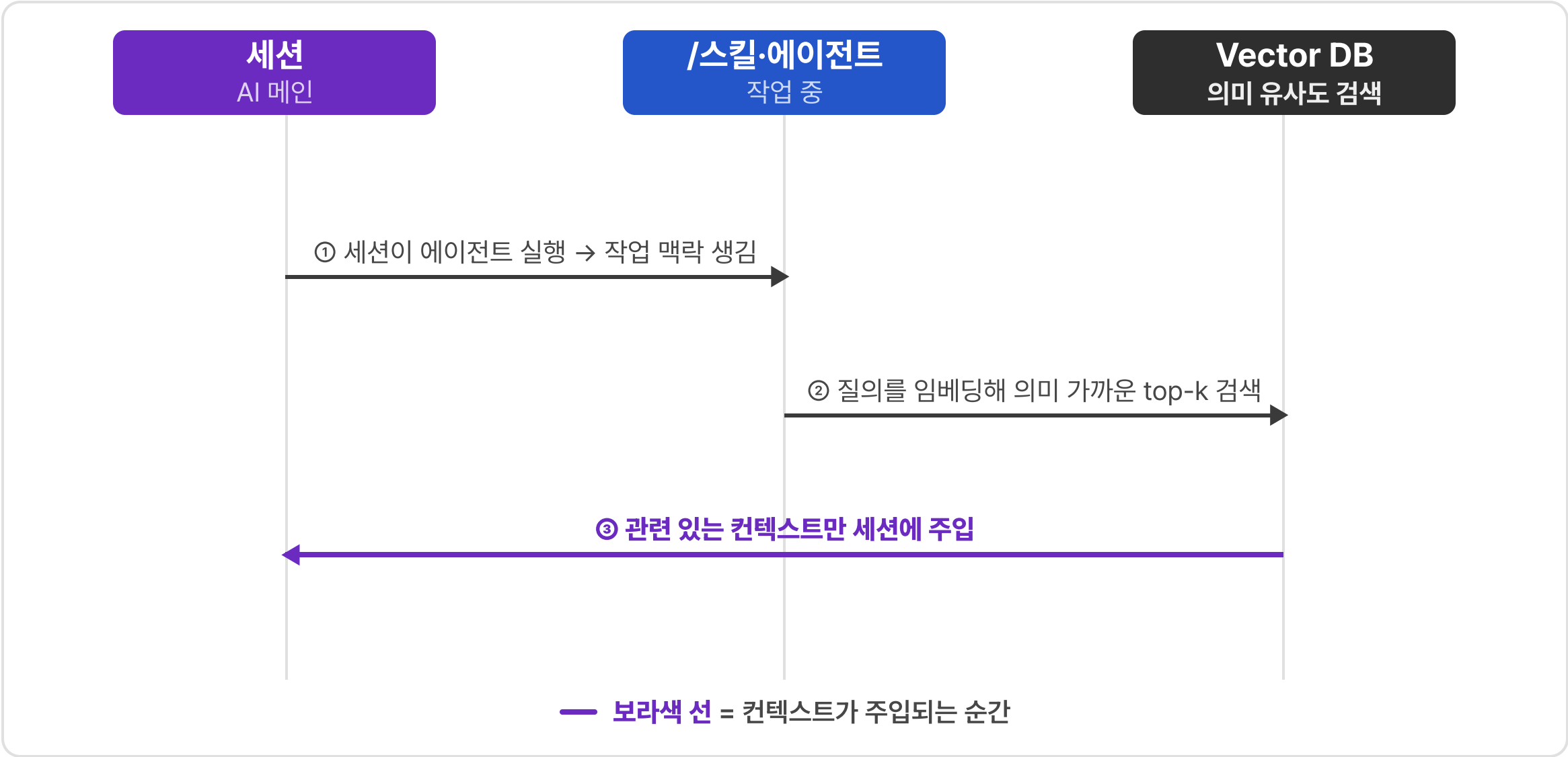
훅이 매번 실행돼 즉시·확실하게 끼워 넣음 (rules와 동일한 특성)

단점

정교한 구분 어려움

정적이라 상황에 따른 세밀한 구분이 힘들

의도 전달 · RAG (Vector + pre-hook)



“ 임베딩으로 의미가 가까운 컨텍스트만 검색 해 혹은 주입하는, 구현이 가장 복잡.무거운 방식.

장점

필요한 것만 최소 주입

관련 있는 컨텍스트만 골라 넣어 대용량 환경에 적합

단점

구축.운영 비용 큼

벡터 인프라를 갖추고 유지하는 데 많은 비용이 듭

의도 전달 · AI가 이해했는지 확인



01
?

질문하기

LLM에게 질문하고 출처를 요구

02


A/B 테스트

컨텍스트 있을 때(A) · 없을 때(B) 각각 스킴을
실행하고 차이를 비교

03


직접 로깅

hook(script) 기반이면 로그 등으로 직접 확인

의도 전달 · 정리



컨텍스트 엔지니어링 등장 배경



핵심

컨텍스트 엔지니어링의 핵심 중 하나는
의도/정보의 전달 체계를 만드는 것



Q&A

주제 - 컨텍스트 엔지니어링과 의도 전달

내 OS 컨텍스트 체계 설계

PRACTICE

나만의 지침을 컨텍스트로 만들고
주입해 실제 활용 확인  **20분**

 **목표** 컨텍스트 체계를 **설계**하고, 현업의 더 복잡한 컨텍스트를
구조적으로 보는 관점 확보

01 내 OS에 어떤 컨텍스트가 필요한지 설계해보기

02 나만의 지침을 간단히 담은 컨텍스트(md) 파일 1-2개 이상 작성

03 컨텍스트 주입 방법을 결정하고 적용해보기

04 스킬·에이전트에 주입한 뒤 정보를 실제로 활용하는지 확인

REVIEW

컨텍스트 체계 설계 나누기

 **10분**

1인 2분 다른 사람의 의견도 들을 수 있도록 **배려** 🙏

SHARE 01

내 OS에 어떤 컨텍스트가 필요하다고 판단했는지

SHARE 02

어떤 방식으로 컨텍스트를 주입했는지



Q&A

주제 - 컨텍스트 체계 설계

DAY 2

경험 공유

컨텍스트 체계의 진화

경험 공유 · 컨텍스트 체계의 진화 (3단계)



STEP 1

컨텍스트 체계가 없던 시기

- 매번 프롬프트로 설명
- CLAUDE.md 활용 시작
- "최선은 아니다"



STEP 2

LLM 동적 로딩의 시기

- CLAUDE.md 매핑 테이블
- 스킬 앞 단계 동적 로딩
- **느림 (10-30초)**
- 누락 (비결정성·매핑 회피)



STEP 3 **현재**

AI First로 관점 전환

- **AI가 일하기 좋은 환경** 지향
- 팀·도메인·전문성 3종 컨텍스트
- **혹으로 자동 주입**
- retrospect·ssot 갱신 보장

경험 공유 · AI First



기존

사람 친화 우선



AI FIRST

AI가 일하기 좋은 환경

AI가 일하기 좋은 환경이 더 중요하다고 판단 · 예. 사람의 가독성을 포기할 수 있음



목표

AI가 **모든 것**을 알게 하기



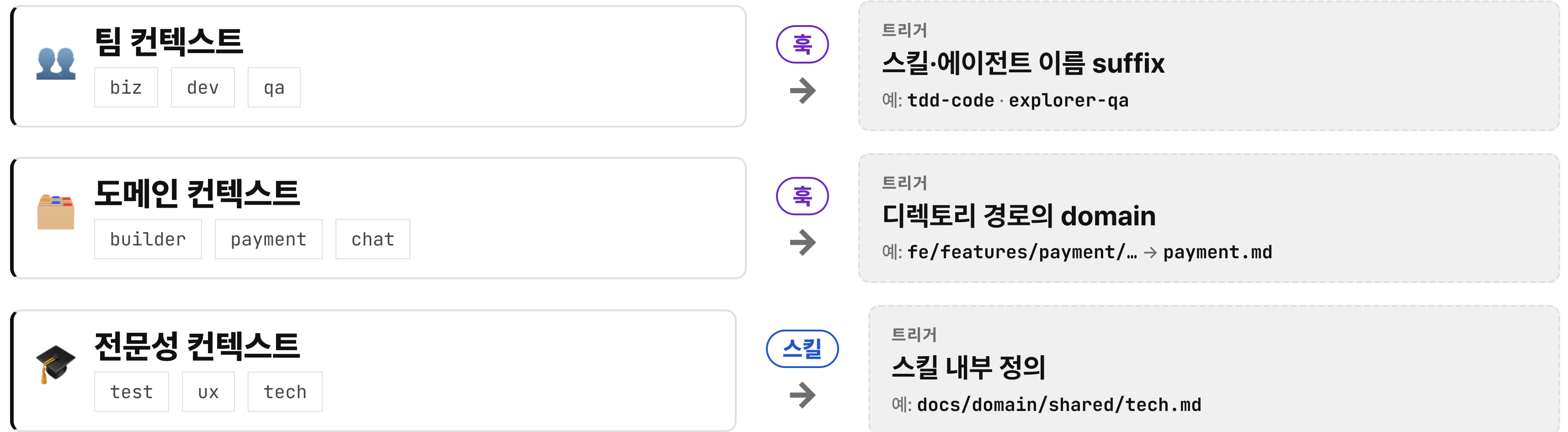
방식

OS에서 업무하면 **의도와 정보가 자동으로** 쌓이는 구조 만들기

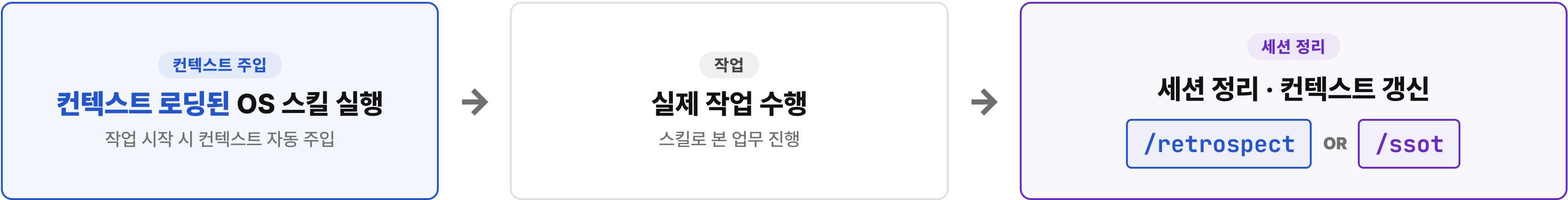
경험 공유 · 컨텍스트 3분류 → 자동 주입



분류된 3종 컨텍스트는 각자의 트리거에 매핑되어 자동 주입됨



경험 공유 · 컨텍스트 갱신 루프



/retrospect

세션 마지막에 호출 - 세션 기록을 보고 의도 **drift** 검사·갱신

- < 빠름
- 정확도 낮음



/ssot

새 의도·정보가 있을 때 **심층 분석으로 문서 갱신**

- 정확
- 느림

경험 공유 · 개선된 점 (전 → 후)



 **컨텍스트 3분류**
팀 · 도메인 · 전문성

 **훅·스킬 자동 주입**
이름 · 경로 · 스킬 트리거 매핑

 **갱신 루프**
/retrospect · /ssot

↓
그 결과

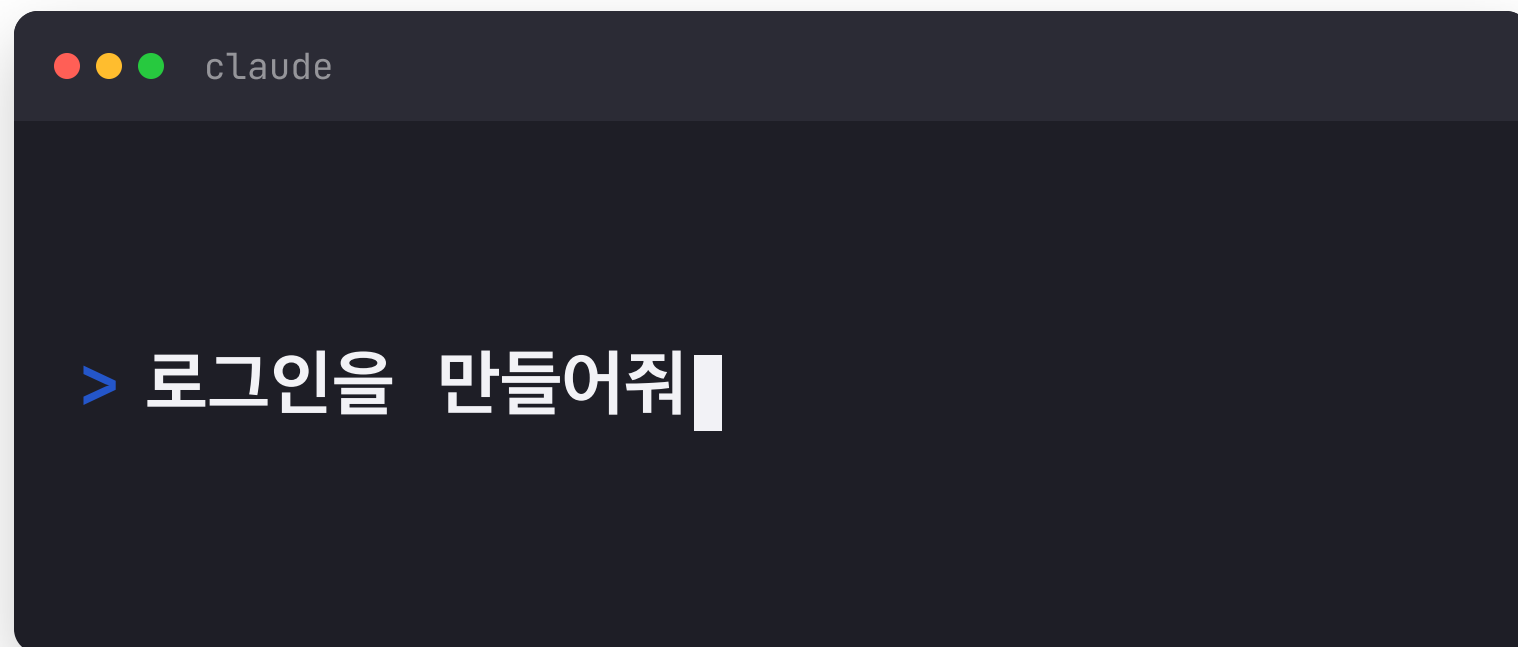
10-30초 소모	→	ms 단위 속도
비결정적	→	결정적
테스트 작성 어려움	→	유닛 테스트 작성 가능
갱신 누락 · 수동	→ ↳ 루프	작업하며 자동 갱신

DAY 2

의도 추출

interview로 모호함을 구체화

의도 추출 · 모호함 문제



이런 프롬프트가 있다고 가정

🤔 이런 모호함이 있다

01 네이버? 구글? 페이스북? 아니면 자체 로그인?

02 이름·전화번호 등 어떤 사용자 정보가 필요한가?

03 로그인 지속시간은 어떻게 되는가?

AI가 때로 적극적으로 물어보기도 하지만, 모호한 요청일 때의 결과는 대부분 복불복

의도 추출

• /interview 로 의도 추출



모호한 의도를 구체화 하는 것



소크라틱 문답법의 영향을 받은 것으로 추정

유명 오픈소스는 모두 interview류 스킬을 가지고 있음

ouroboros

gstack

superpower



Q&A

주제 - 의도 추출(interview)

내 OS에 맞는 interview 스킬 만들기

PRACTICE

모호함을 구체화하는 나만의 interview 스킬 설계·제작

 25분

목표 만들기

코드는 AI가 만드는 시대, **나만의 철학을 투영하는** 인터뷰 스킬

01 모호함을 구체적으로 바꾸는 방법 설계 (여기에 시간 투자 권장)

- 구체성으로 변환하는 핵심 방법론은? (질문 10개? 모호함 측정? 등)
- 구체화할 영역은? UX? 아키텍처? 구현 가능성?
- 클로드 코드와 논의 혹은 오픈소스 방식 참고 추천

02 위 설계를 토대로 interview 스킬 만들어보기

03 interview 스킬로 모호한 생각 구체화해보기

예: 인터뷰 후 스킬 생성

REVIEW

interview 스킬 나누기

 15분

1인 3분

다른 사람의 의견도 들을 수 있도록 **배려** 🙏

SHARE 01

모호함을 구체적으로 바꾸기 위해 어떤 방법을 채택했나요?

SHARE 02

프롬프트로 스킬을 만들 때와, interview를 통해 만들 때의 차이는 어땠나요?



Q&A

주제 - 의도 추출과 interview 스킬

DAY 2

컨텍스트 최적화

짧을수록 유리하다 - 최적화 전략 6가지

컨텍스트 최적화 · 짧은 컨텍스트가 유리한 이유



01 입력 토큰이 늘수록 모델 성능이 점진적으로 저하

Context Rot

02 토큰 비례 비용·지연도 증가 - 품질·속도·비용 모두 손해

03 Opus가 1M 컨텍스트를 제공하는 것이 나쁜 디자인이라는 견해도 있음

결론 - 짧은 컨텍스트가 성능 안정성에 유리 [1] Context Rot 연구 (Chroma, 2025)

컨텍스트 최적화 · CLAUDE.md 불필요 제거



CLAUDE.md는 모든 세션에 기본 로딩 - "정말 모든 상황에서 필요한가?"를 통과한 항목만 남김

CLAUDE.md

전체

- ✓ 항상 필요한 규칙
- ✗ ~~가끔만 쓰는 지침~~
- ✗ ~~특정 상황 전용 메모~~
- ✓ 핵심 컨벤션
- ✗ ~~한 번 쓰고 남은 내용~~

"모든 상황에서
필요한가?"



CLAUDE.md

슬림

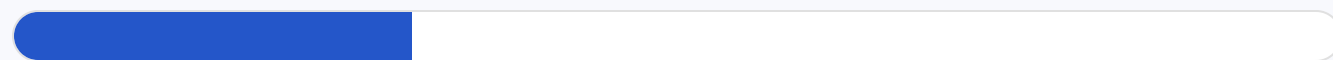
- ✓ 항상 필요한 규칙
- ✓ 핵심 컨벤션

컨텍스트 최적화 · 에이전트 위임 적극 활용



에이전트는 컨텍스트가 독립적 - 큰 검색·리뷰를 위임하면 토큰을 흡수해 메인을 가볍게 유지함

메인 컨텍스트



가볍게 유지

큰 검색·리뷰 위임 →

← 결과 요약만 반환

에이전트



독립 컨텍스트에서 큰 작업 처리

⚠ 단점 위임·요약 과정에서 맥락이 유실되고 Spawn, 컨텍스트 로딩 등의 추가 비용 발생

컨텍스트 최적화 · 컨텍스트 Lazy 로딩



구분	전역 주입	LAZY 로딩 권장
로딩 시점	항상 전부	필요한 시점만 (LLM 판단)
컨텍스트량	항상 가득	필요한 것만 최소
트레이드오프	단순하지만 매번 비용	Lazy 로딩 ↔ 느림·비결정

👉 전역으로 다 주입하지 말고, **필요한 것만 필요할 때**

⚠️ 단점 느리고, 무엇을 로딩할지 LLM 판단에 의존해 비결정적

컨텍스트 최적화 · 출력 토큰 줄이기



RTK Token Savings (Global Scope)

Total commands: 80860
Input tokens: 508.5M
Output tokens: 275.6M
Tokens saved: 233.2M (45.9%)
Total exec time: 7768m5s (avg 5.8s)
Efficiency meter:  45.9%

By Command

#	Command	Count	Saved	Avg%	Time	Impact
1.	rtk vitest run	4980	80.7M	75.3%	27.2s	
2.	rtk lint eslint --fix .	1	41.7M	100.0%	2m41s	
3.	rtk read	10558	40.5M	23.2%	1ms	
4.	rtk grep	8506	31.9M	20.7%	375ms	
5.	rtk:toml ps aux	192	11.0M	98.3%	176ms	
6.	rtk:toml ps -ef	90	5.4M	98.9%	147ms	
7.	rtk ls -la /Users/cha ...	28	1.7M	96.6%	148ms	
8.	rtk find	2730	1.4M	64.9%	67ms	
9.	rtk tsc --noEmit	542	1.3M	39.1%	20.1s	
10.	rtk git diff main..HE ...	1	1.2M	98.8%	238ms	

60-90% ↓

"1줄로 답해" 같은 짧은 응답 강제

RTK 같은 CLI proxy로 git·ls 등 일반 명령 토큰 절감 [2] RTK (Rust Token Killer) · CLI proxy

⚠ 단점 과하게 줄이면 근거·중간 추론이 사라져 단서 추적이 어려워짐

컨텍스트 최적화 · 긴 작업은 phase로 분해



긴 작업을 한 세션에 몰면 컨텍스트가 쌓임 - 계획을 파일에 박제한 뒤 **phase**를 나눠 각각 새 세션으로 실행

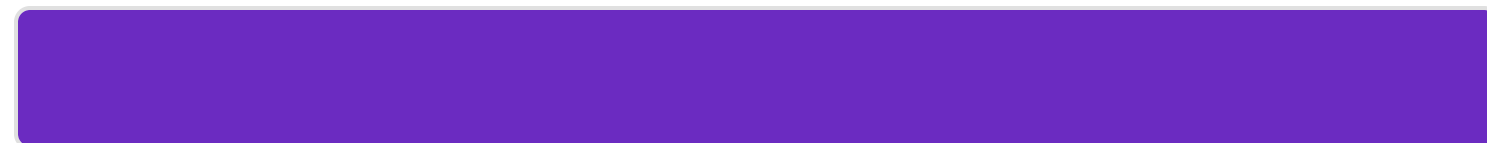


△ 단점 Spawn, 컨텍스트 로딩 비용 반복 발생, phase 경계에서 컨텍스트 유실로 성능 저하

컨텍스트 최적화 · `/clear` 습관화

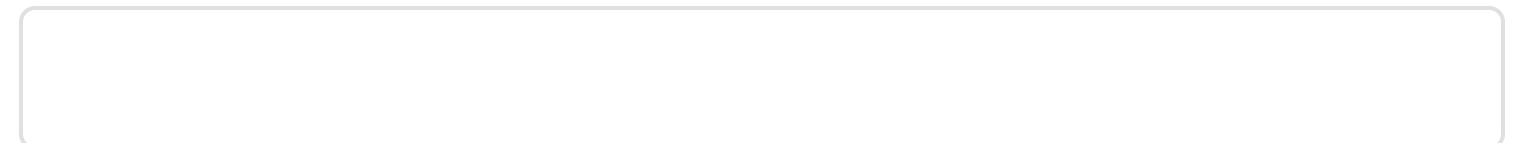


작업이 끝나면 즉시 `/clear` - 쌓인 컨텍스트를 비우고 다음 작업을 깨끗하게 시작



작업 종료 시 - 가득 찬 컨텍스트

`/clear`



리셋 - 다음 작업을 깨끗하게

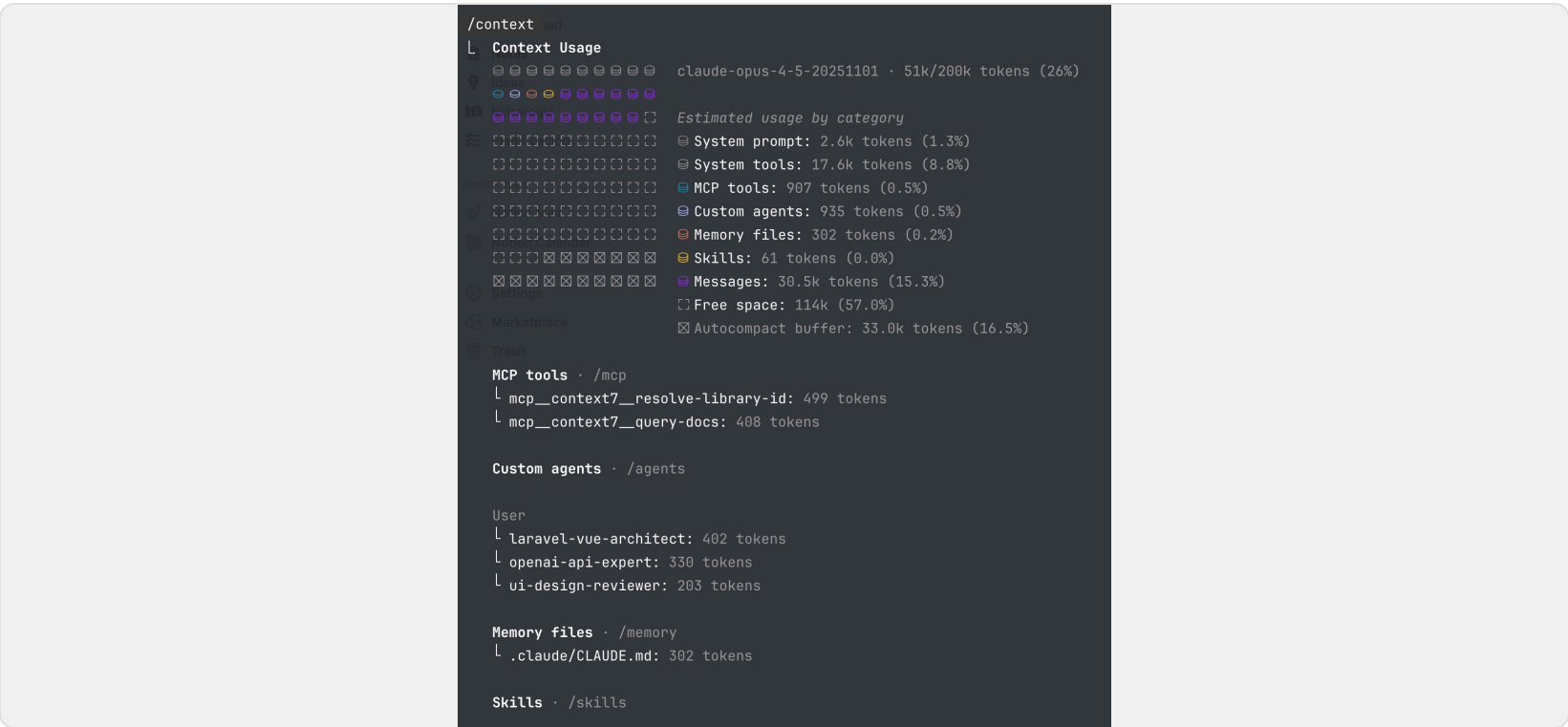
⚠ 단점 너무 일찍 비우면 직전 작업 맥락을 잃어 다시 쌓아야 함

컨텍스트 최적화 · 컨텍스트 측정하기



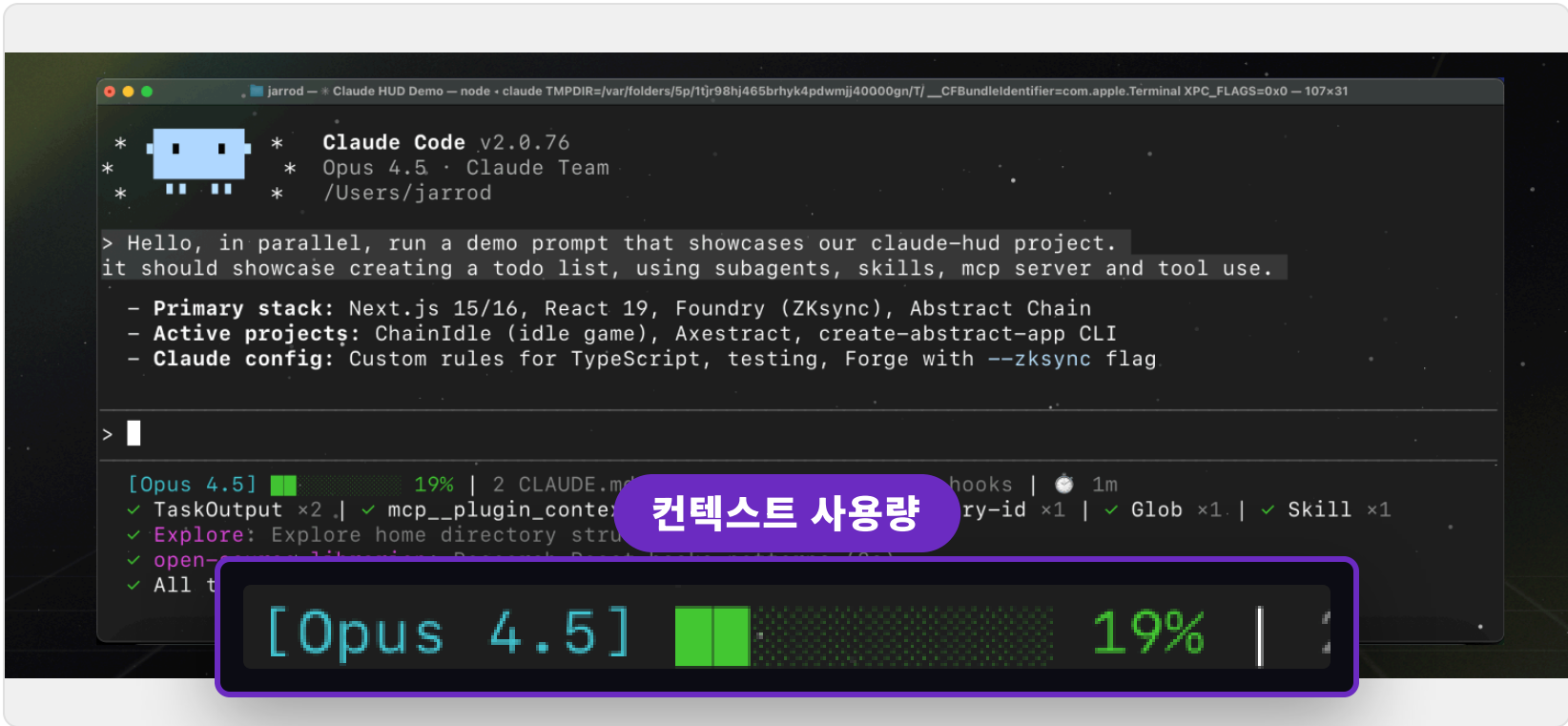
/context

세션의 컨텍스트 구성·사용량을 한눈에 측정



claude-hud

OSS HUD로 컨텍스트량을 **상시 표시**





Q&A

주제 - 컨텍스트 최적화

컨텍스트 최적화

PRACTICE

측정 → 최적화 → 재측정으로 전·후 비교  20분

 목표 컨텍스트를 제어하고, 이후 문제를 만났을 때 대응할 소양 쌓기

01 /clear 후 바로 /context 호출해서 기본 컨텍스트랑 측정

02 그 다음 특정 스킬 실행하고 컨텍스트 측정해보기

03 최적화 해본 후 다시 측정하고 전·후 비교해보기

REVIEW

컨텍스트 최적화 + 일일 회고

 15분

1인 3분 다른 사람의 의견도 들을 수 있도록 배려 🙏

SHARE 01

어떤 전략으로 토큰 최적화를 접근했는지 나누기

SHARE 02

측정을 어떻게 했고, 정량적으로 어느 정도 차이를 만들었는지 나누기

SHARE 03

컨텍스트 엔지니어링을 앞으로 내 OS에 활용할 방향 나누기

오늘 완료한 것



컨텍스트 엔지니어링 체험

무엇을 / 어떻게 / 준수 확인 - 3대 고려 요소를 직접 설계



interview 스킬 제작·체험

의도 추출에 유용하고 OS의 든든한 기반이 될 스킬



컨텍스트 최적화 체험

6가지 전략으로 컨텍스트 최적화, 전·후 비교

이제 할 수 있는 것

현업 환경에서 컨텍스트의 중요성을 설명하고 기초 설계를 할 수 있음

컨텍스트 오염

01 기획·의도가 바뀌며 컨텍스트 문서가 **Out-of-Date**

02 옛 기준의 의도·정보를 활용해 **의도치 않은 결과** 발생

03 요즘은 이를 **drift**라고 부름

클로드 코드가 자주 쓰는 용어

루프 미리보기 (3일차 주제)

컨텍스트 오염을 방지할 방법이 필요함



👉 3일차에는 이런 루프(루프 엔지니어링)를 다룰 예정

내 OS에 컨텍스트 체계 심기

💡 강사 권장

네이버가 컨텍스트 관련 공용 플랫폼을 만들었듯, 컨텍스트는 OS 체계가 커질수록 중요한 인프라가 됩니다. 회사와 달리 마음껏 제어할 수 있는 내 OS에 컨텍스트 체계를 직접 설계해보시면 좋겠습니다.

완료 조건

필수

- 1

컨텍스트 파일 3개 + 자동 주입

컨텍스트(md) 파일 3개 이상 작성 + 내 OS 스킬·서브에이전트에 자동 주입되게 연결
- 2

주입 A/B 동작 비교

주입됐을 때 vs 안 됐을 때 스킬 동작을 클로드 내부 도구 /skill-creator로 비교
- 3

컨텍스트 체계 도식화

내 컨텍스트 체계를 시각적으로 도식화해 파일로 남기기 (1P 추천)

도전

- 1

주입 검증 테스트

컨텍스트 주입이 올바른지 확인하는 테스트 제작 (버그 방지 목적)
- 2

최적화 + 정량 비교

컨텍스트 체계를 다양한 방법으로 최적화하고 정량적으로 비교

과제 TIP

 PR을 할 때 접근 방식 · 설계 의도 · 궁금한 점을 함께 적어주시면 더 양질의 리뷰를 드릴 수 있습니다!

 4주 동안 OS를 만들고 · 사용하고 · 개선 할수록 강의에서 더 많은 것을 얻어가실 수 있습니다!